Qualcomm Technologies, Inc.

AudioReach on Android

80-VN500-19 Rev. AA

November 16, 2020

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:56 GMT hijiasheng@longcheer.com

For additional information or to submit technical questions, go to: https://createpoint.qti.qualcomm.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Restricted Distribution: Not to be distributed to anyone who is not an employee of either Qualcomm Technologies, Inc. or its affiliated companies without the express approval of Qualcomm Configuration Management. Distribution to anyone who is not an employee of either Qualcomm Incorporated or its affiliated companies is subject to applicable confidentiality agreements.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark of Qualcomm Incorporated, registered in the United States and other countries. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer (“export”) laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2020 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AA	November 2020	Initial release

Note: There is no Rev. I, O, Q, S, X, or Z per Mil. standards.

Qualcomm
Confidential - May Contain Trade Secrets
2022-08-10 09:38:56 GMT
nijiasheng@longcheer.com

Contents

1 Introduction 4

1.1 Purpose 4
1.2 Conventions 4
1.3 Technical assistance 4

2 Architecture 5

3 AudioReach implementation 7

3.1 Representation of audio use cases 8
3.1.1 Graphs 8
3.1.2 Subgraphs 8
3.1.3 Keys and key vectors 8
3.2 Module identification and configuration 10
3.3 Calibration key vectors 11
3.4 TinyALSA and plugins 13
3.5 AGM 15
3.5.1 Session object 16
3.5.2 Graph object 16
3.5.3 Device object 16
3.5.4 AGM service 17
3.5.5 Tags in AGM 18
3.6 AML 19
3.7 QAL 19

4 Audio use cases 21

4.1 Playback 21
4.1.1 Graph 21
4.1.2 High-level call flow 22
4.1.3 Logcat logs 23

A References 32

A.1 Acronyms and terms 32

Qualcomm Confidential – May Contain Trade Secrets
2022-06-10 09:38:56 GMT
nijsheng@long-cheer.com

1 Introduction

1.1 Purpose

This document provides an introduction to AudioReach™, Qualcomm Technologies, Inc. (QTI)'s new data-driven audio architecture developed to reduce overall development effort for customers and enable faster time-to-market for new features and capabilities at a lower cost. This document provides an introduction to the AudioReach software architecture as well as an explanation of the various modules and audio drivers in AudioReach. It also provides an overview of the signal processing framework (SPF) that is supported by the ADSP in the AudioReach architecture.

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `cp armcc armcpp`.

Code variables appear in angle brackets, for example, `<number>`.

Commands to be entered appear in a different font, for example, **`copy a:*.* b:`**.

Button and key names appear in bold font, for example, click **Save** or press **Enter**.

Shading indicates content that has been added or changed in this revision of the document.

1.3 Technical assistance

For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at https://createpoint.qti.qualcomm.com/.

If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

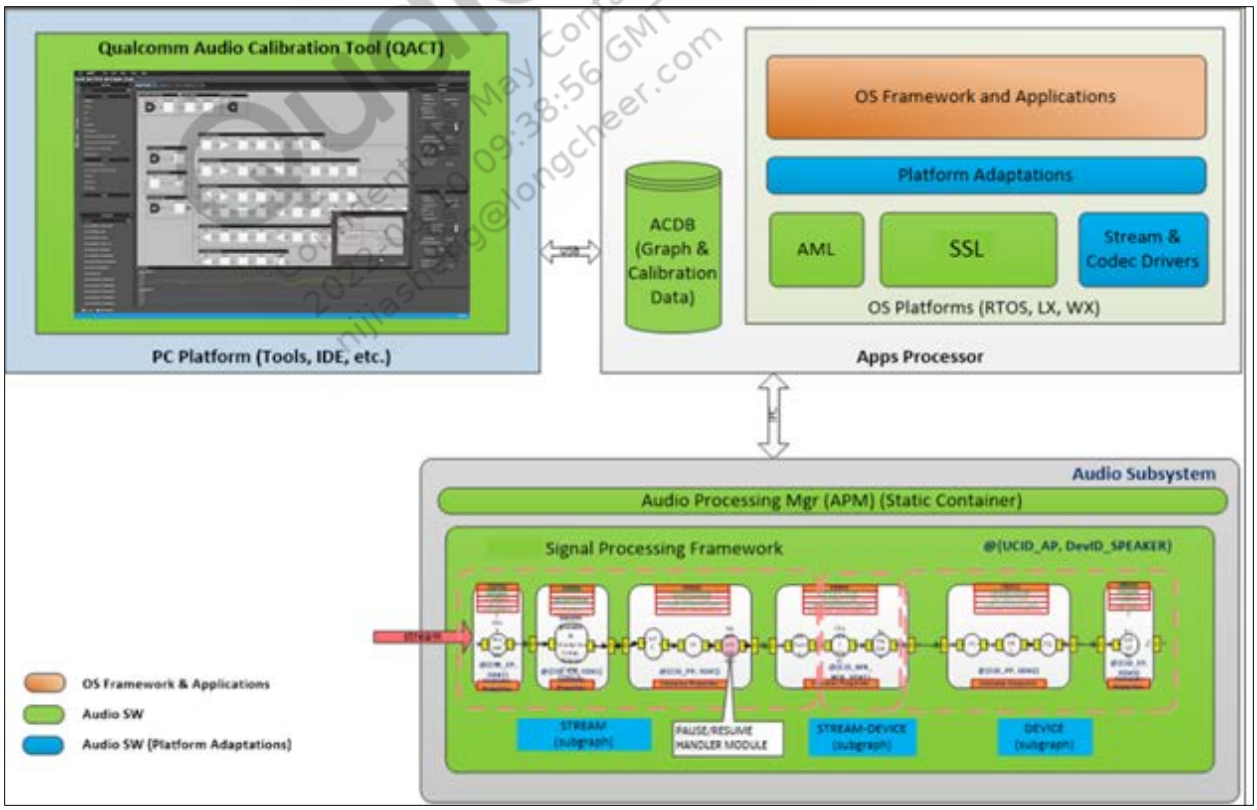
Qualcomm Confidential - May Contain Trade Secrets
2022-08-10 06:36:56 GMT
nijiashengzhenzhen@163.com

2 Architecture

AudioReach provides modularized, extensible, platform-agnostic audio signal processing framework that supports a broad spectrum of audio and voice use cases with the flexibility to scale up or down to richer or more limited processing environments. AudioReach is a complete package of software components, development kits, and tools to design, implement, and validate end-to-end audio use cases across multiple platforms.

The QACT™ platform (packaged with AudioReach) is a PC-based tool that provides a and user-friendly environment for development, configuration, and tuning that allows audio software and tuning engineers to focus on creating, tuning, and optimizing differentiating IP.

The following diagram shows the AudioReach high-level architecture.



As shown in the diagram:

- The AudioReach architecture supports graph-based use case definitions

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

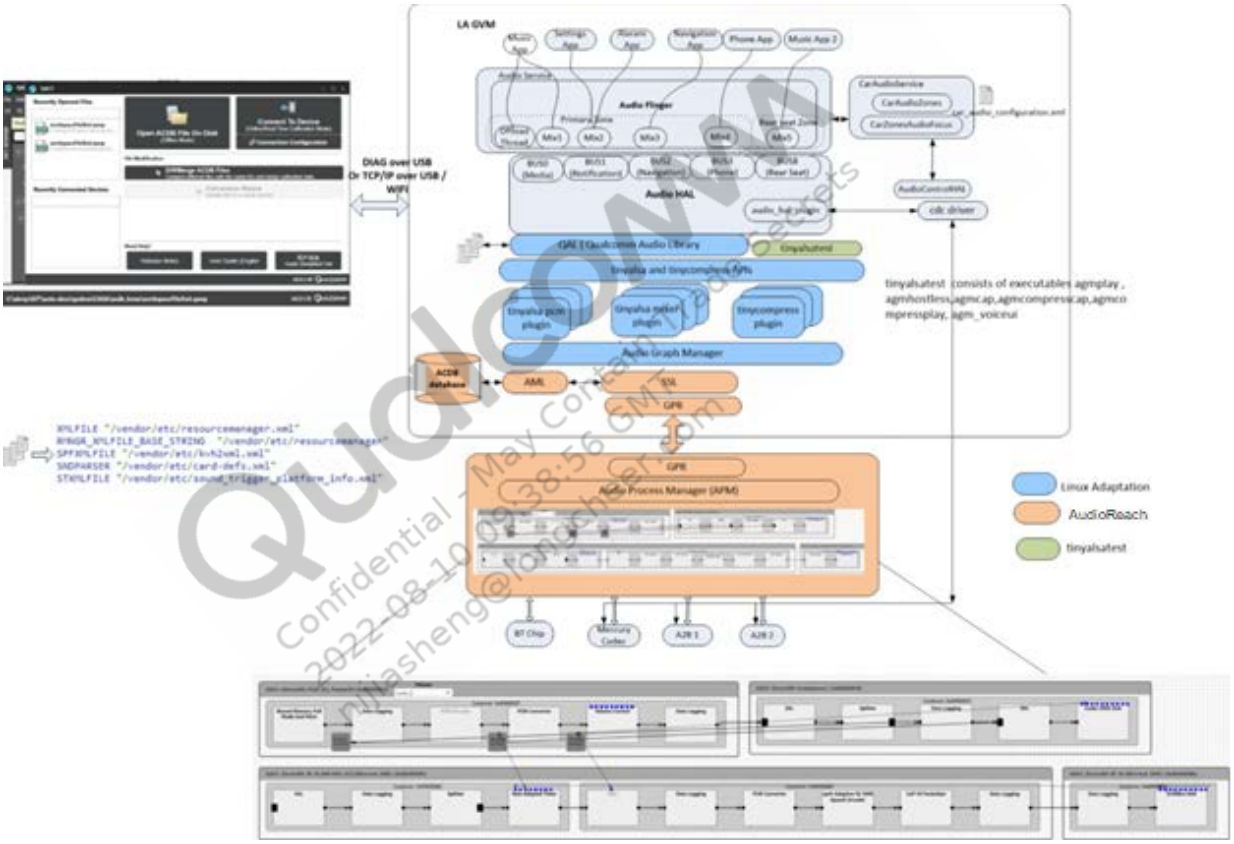
- The software running on the audio subsystem, which on QTI chipsets represents the application DSP (ADSP), implements the SPF and the audio processing manager (APM), a static container that exchanges commands with the host processor via inter processor communication (IPC).
- The SPF can construct a use case graph consisting of subgraphs containing one or more modules, each of which has a source and sink endpoint (EP). Each module in a sub graph has a defined function, such as:
  - Logging PCM data
  - Splitting the incoming data stream into different streams
  - Mixing more than one data stream into a single data stream
  - Applying a specific PCM processing algorithm
  - Configuring a hardware EP
  - Exchanging data with the host processor
- The APM is responsible for creating, configuring, and destroying use case graphs based on the control and data commands it receives from the graph service layer (GSL)) that runs on the HLOS.

  There is no intelligence in the GSL regarding the various use case graphs. All the information about the various use cases that can be instantiated by the SPF in the ADSP is present in the audio calibration data base (ACDB). The ACDB is created by the apps processor memory by parsing a given ACDB file.
- The audio manager library (AML) is responsible for parsing the .acdb file present in the filesystem and creating the ACDB database. The AML exposes APIs that are called by the SSL to receive the use case graph information from the ACDB. The SSL then creates the graph packet router (GPR) packet, associates a payload with it, and sends it to the ADSP via the GPR driver (not shown in the diagram). The GPR driver communicates with the ADSP via IPC.
- The platform adaptation layer shown in the diagram is different from one platform to another. This layer is responsible for translating the use case information triggered from application layer to information that can be passed to the SSL layer so the corresponding use case graph can be realized in the ADSP.
- QACT provides a graphical interface the system designer can use to create various use case graphs that be saved in an ACDB file. QACT can import XML files that define the module interface (modules that constitute a subgraph) as well as XML configuration files used to define subgraphs, inter-connectivity between graphs, etc.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3 AudioReach implementation

The following shows the Android audio architecture with AudioReach.



In this diagram:

- The Android adaptation layer is implemented between AudioReach and the Android audio HAL. This layer exposes the Qualcomm audio library (QAL) APIs that are called by the audio HAL.
- The QAL module calls TinyALSA APIs that internally call the corresponding TinyALSA plugin APIs and pass the control/data to the audio graph manager (AGM). The AGM decodes the passed configuration parameter by calling TinyALSA mixer plugin APIs. It also handles the data passed from the upper layers and propagates data received from the ADSP via SSL back to these upper layers. SSL interacts with the AML to get the use case graph information and all parameters required to instantiate the use case graph in the ADSP.
- The GSL passes the information to the APM static service container in the ADSP so that the use case can be started in the ADSP.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.1 Representation of audio use cases

Each audio use case is represented as a graph consisting of subgraphs of a specific type. Each subgraph consists of one or more functional software blocks (referred to as modules) that perform a specific function.

3.1.1 Graphs

A graph is logical representation of the modules connected together in order to realize a specific audio or voice use case.

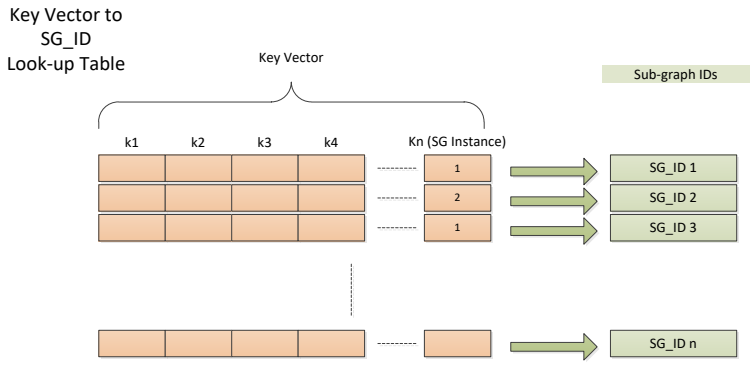
3.1.2 Subgraphs

Subgraphs are introduced so that portions of the graphs can be manipulated independently when switching between use cases. This independent manipulation allows for lower latencies, minimal data loss, and a lesser number of control operations when switching between two use cases. The grouping is also beneficial when there are a common group of modules between the use cases. This group of modules can be grouped together as a subgraph.

3.1.3 Keys and key vectors

Keys are used by the GSL as an index or indices to retrieve subgraph IDs (SG_ID) and associated blobs from ACDB. GSL uses this SG_ID to then query the graph definition, that is, modules and the edge information between modules from ACDB. Similarly, keys are also used to retrieve calibration data from ACDB for a given subgraph.

A key vector is represented by a set of multiple key value pairs as shown in the following figure. The graph or system designer associates a set of unique <keys> and <values> when creating a subgraph from the UI canvas. QACT automatically assigns a unique SG_ID for each subgraph that is created by the system designer. A look-up table (LUT) is built by the tool, populating all the keys that map to SG_IDs.



GSL does not define specific keys – It is up to the platform adaptations to define the keys that are required to get subgraphs and blobs. GSL does not interpret the keys – Instead, it uses them to retrieve the subgraph IDs and blobs.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

Example

PCM low latency playback is represented as a graph key vector GKV:

<StreamRX:PCM_LL_Playback , DevicePP_RX:Audio_MBDRC, Instance:Intance_1,DeviceRX:Speaker>

Select	GKV	SubGraphs
◉	<StreamRX:PCM_LL_Playback, DeviceRX:Speaker, Instance:Instance_1, DevicePP_Rx:Audio_MBDRC>	0xB00000D3 0xB000002D 0xB000002E 0xB0000023
○	<StreamRX:PCM_LL_Playback, DeviceRX:Headphones, Instance:Instance_1, DevicePP_Rx:Audio_MBDRC>	0xB0000014 0xB0000026 0xB0000027 0xB0000023
○	<StreamRX:PCM_LL_Playback, DeviceRX:USB_Rx, Instance:Instance_1, DevicePP_Rx:Audio_MBDRC>	0xB0000097 0xB0000096 0xB000009D 0xB0000023
○	<StreamRX:PCM_LL_Playback, DeviceRX:HDMI_Rx, Instance:Instance_1, DevicePP_Rx:Audio_MBDRC>	0xB0000093 0xB0000092 0xB00000A1 0xB0000023
○	<StreamRX:PCM_LL_Playback, DeviceRX:BT_Rx, Instance:Instance_1, DevicePP_Rx:Audio_MBDRC, BtProfile:SCO>	0xB0000019 0xB0000061 0xB0000065 0xB0000023
○	<StreamRX:PCM_LL_Playback, DeviceRX:BT_Rx, Instance:Instance_1, DevicePP_Rx:Audio_MBDRC, BtProfile:SCO, BtFormat:SWB>	0xB00000B6 0xB00000B5 0xB0000061 0xB0000065 0xB0000023
○	<StreamRX:PCM_LL_Playback, DeviceRX:BT_Rx, Instance:Instance_1, DevicePP_Rx:Audio_MBDRC, BtProfile:A2DP, BtFormat:GENERIC>	0xB000006C 0xB000006B 0xB000006A 0xB0000070 0xB0000023
○	<StreamRX:PCM_LL_Playback, DeviceRX:BT_Rx, Instance:Instance_1, DevicePP_Rx:Audio_MBDRC, BtProfile:A2DP, BtFormat:LDAC>	0xB000006C 0xB00000A9 0xB000006A 0xB0000070 0xB0000023
○	<StreamRX:PCM_LL_Playback, DeviceRX:BT_Rx, Instance:Instance_1, DevicePP_Rx:Audio_MBDRC, BtProfile:A2DP, BtFormat:APTX_ADAPTIVE>	0xB000006C 0xB00000AB 0xB000006A 0xB0000070 0xB0000023

As shown, QACT assigns a subgraph ID (for example, 0xB00000D3, 0xB000002D, etc.) based on the subgraph's key vector. Refer to Section 4.1 for more information on how subgraph key vectors are assigned a subgraph ID.

The following table lists the GKVs in mm-audio/casa/acdbdata/inc/kvh2xml.h.

Enum in kvh2xml.h	ID	QACT string	Subgraph type
STREAMRX	0xA1000000	StreamRX	STREAM
STREAMTX	0xB1000000	StreamTX	STREAM
DEVICERX	0xA2000000	DeviceRX	DEVICE
DEVICETX	0xA3000000	DeviceTX	DEVICE
DEVICEPP_RX	0xAC000000	DevicePP_Rx	DEVICE_PP
DEVICEPP_TX	0xAD000000	DevicePP_Tx	DEVICE_PP
STREAMPP_RX	0xAF000000	StreamPP_RX	STREAM_PP
STREAMPP_TX	0xB0000000	StreamPP_TX	STREAM_PP

Based on the value of the GKV, each subgraph consists of various modules. The subgraph type IDs are present in /gsl/api/gsl_subgraph_platform_driver_props.xml.

```
<!-- Copyright (c) 2019 Qualcomm Technologies, Inc.             -->
 <!-- All Rights Reserved.                                        -->
 <!-- Confidential and Proprietary - Qualcomm Technologies, Inc. -->
 <DRIVER_PROP_DEF versionMajor="0x0" versionMinor="0x0" versionBranch="0x0"
versionSubBranch="0x0" title_date="08/01/2018">
  <PROP id="0x08000010" name="SUB_GRAPH_TYPE" description="">
    <CONFIG_ELEMENT name="sg_type" type="uint32_t" byteSize="0x4"
byteOffset="0x0" min="0x1" max="0x5" default="0x1" description="">
      <RANGE_LIST>
        <RANGE name="STREAM" val="0x1"/>
        <RANGE name="DEVICE" val="0x2"/>
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
          <RANGE name="STREAM_DEVICE" val="0x3"/>
          <RANGE name="STREAM_PP" val="0x4"/>
          <RANGE name="DEVICE_PP" val="0x5"/>
        </RANGE_LIST>
      </CONFIG_ELEMENT>
    </PROP>
  </DRIVER_PROP_DEF>
```

3.2 Module identification and configuration

A given use case may require updating the configuration of one or more modules in a graph to enable, disable, or configure a certain capability of a module at runtime (UI initiated or otherwise). Since the module (ID and instance ID) implementing this capability may be different across different subgraphs, a mechanism to identify these modules (module IDs and instance IDs) in a generic way is needed.

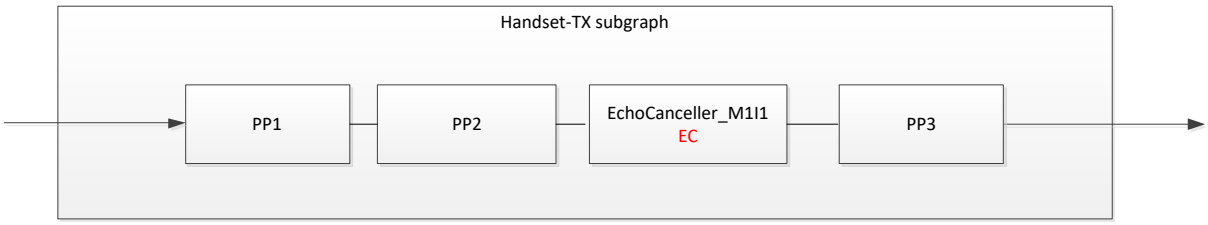
Some use cases that require configuring a module include runtime-controllable audio effects such as noise suppression, echo cancellation, virtualizer, equalizers, ANC, HD voice, slowtalk, volume, etc.

Tags

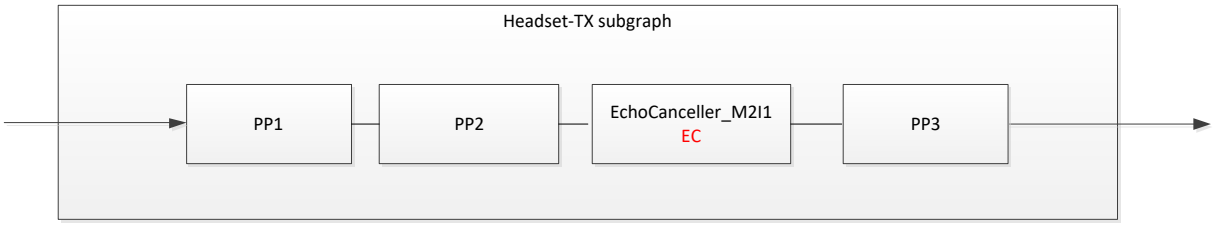
A tag is an identifier set to identify runtime controllable capabilities of one or more modules. Tag are needed because, for a given graph, the audio driver software does not know the module ID/instance ID that implements a given runtime controllable capability such as volume. Hence, a generic way for software to communicate with the desired module/instance is needed.

For example, consider the following subgraphs for a handset Tx device and a headset Tx device that includes an echo cancellation module with EchoCanceller_M1I1 and EchoCanceller_M2I1. Both of these echo cancellers have a different implementation of the echo cancellation capability appropriate for the device profiles (handset and headset). Hence, the module IDs are different. However, since both of these modules are enabling echo cancellation capability, both modules can be assigned an EC tag to represent the echo cancellation capability across these subgraphs.

Tag usage allows platform adaptations to address the echo canceller module with a generic tag EC instead of having to know the exact module ID and instance ID in each subgraph.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION



Each platform can publish module tags in a header file. Using the H2XML utility, this header file can be converted to an XML file. This XML file is imported into QACT and the system designer can assign these predefined tags to modules in a subgraph.

A module can be assigned more than one tag to identify different capabilities of the module. A module tag can be assigned to more than one modules if updating certain capability results in updating parameters of one or more modules.

There are different types of runtime controllable features that need to be supported:

- Common platform supported capabilities such as enabling/disabling audio effects, mute, volume, etc.
- QTI value added features such as slowtalk, HD voice, etc.
- Third party custom features with IP concealment requirements. For example, Dolby and DTS prefer not to publish parameter IDs/values via documentation or in a header file.

For all types of runtime controllable features, the GSL client exposes set/get parameter APIs. These runtime parameters need to be translated to one or more module specific parameters in a specific subgraph. Module information and parameters can be stored and retrieved from ACDB database files as a blob in the following format:

<mid1, iid1, param_id_1, param_value_1.. param_id_N, param_value_N>…. <midN, iidN, param_id_1, param_value_1.. param_id_N, param_value_N>

The GSL client supports translating the runtime parameters to the module payload (mid, iid, param ID, param_value1) in the form of paramid/param value in the following two methods:

- Translate runtime parameters to <tag, key> and use this to retrieve the module payload from ACDB data files using ACDB APIs
- Translate runtime params to <tag, key>, retrieve module the payload containing just module ID and instance ID, and generate a binary blob in GSL client.

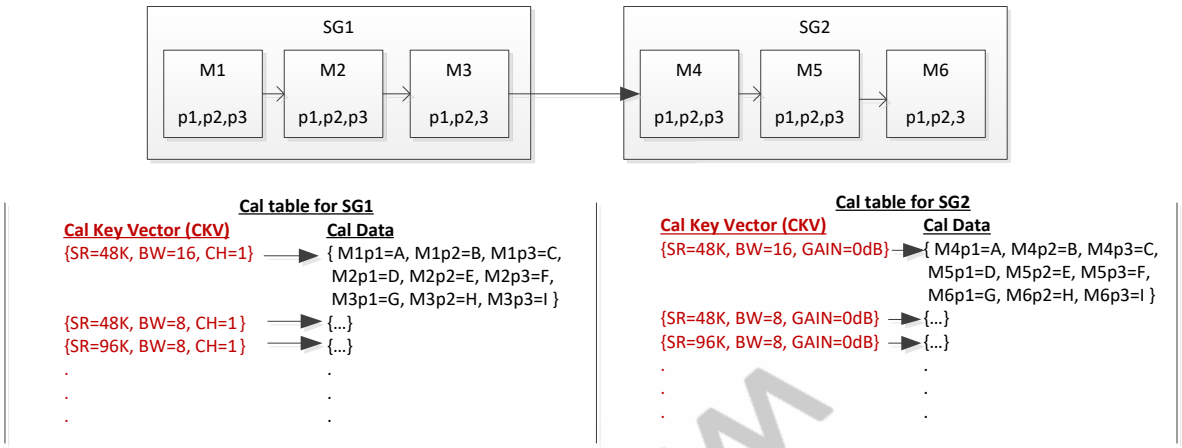
3.3 Calibration key vectors

Once a graph is loaded to the SPF in the DSP, the next step is to push the appropriate calibration for all the modules in the graph. The actual calibration data that should get pushed is highly dependent on runtime parameters such as sample rate, bit width, channels, gains, etc.

The system designer creates a calibration table for each subgraph and arranges the data in a way that it can be indexed by a set of keys and values (or key vector) known as a calibration key vector. At runtime, the GSL client looks up the calibration data using a runtime generated graph key vector.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

The following is an example of calibration data and calibration graph vectors.



Composite Cal Key Vector = { SR=48K, BW=16, CH=1, GAIN=0dB}

The GSL client is responsible for creating the composite cal key vector based on use case information and pushing it to the GSL after the graph is opened. The GSL is responsible for walking through the list of subgraphs, looking up the calibration data for each subgraph, and pushing it to the SPF.

The parameter ID of a module can be tagged to be used in a TKV or calibration key vector (CKV).

PID of module marked as TKV only

If the PID of a module is marked as TKV, it indicates that the configuration data for the PID needs to be constructed dynamically based on the values of each key in the TKV passed. The configuration data, along with the MID, PID, and instance ID are then passed to the ADSP. The configuration/calibration data is not present in the ACDB.

PID of module marked as CKV only

If the PID of a module is marked as CKV, it indicates that the configuration data for the PID is queried from the ACDB based on the values of each key in the CKV passed. The configuration data, along with the MID, PID, and instance ID are then passed to the ADSP. The configuration/calibration data is present in the ACDB.

PID of module marked as both CKV and TKV

If the PID of a module is marked as both CKV and TKV, it indicates that the configuration data can be dynamically generated or queried from the ACDB based on the value of each key in TKV or CKV.

PID of module marked as neither CKV or TKV

When the PID of a module is marked as neither TKV or CKV, it indicates that there no calibration data is present in the ACDB. In this case, there is no way to get the PID, MID, and instance ID by passing any key vector. In such a scenario, the ADSP may have some hardcoded values for the configuration data associated with the PID.

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.4 TinyALSA and plugins

To realize an audio use case in which the software stack supports ALSA or TinyALSA, it is required to know what mixer controls need to be passed to the TinyALSA utility (tinymix) and how to invoke the tinyplay, tinycap, and/or tinyhostless utilities.

TinyALSA supports plugins that interact with the AGM, which in turn calls the SSL. The AGM implements the following plugins that are compiled to make .so files:

- PCM plugin – Compiled as libagm_pcm_plugin.so
- Mixer plugin – Compiled as libagm_mixer_plugin.so
- Compress plugin – Compiled as libagm_compress_plugin.so

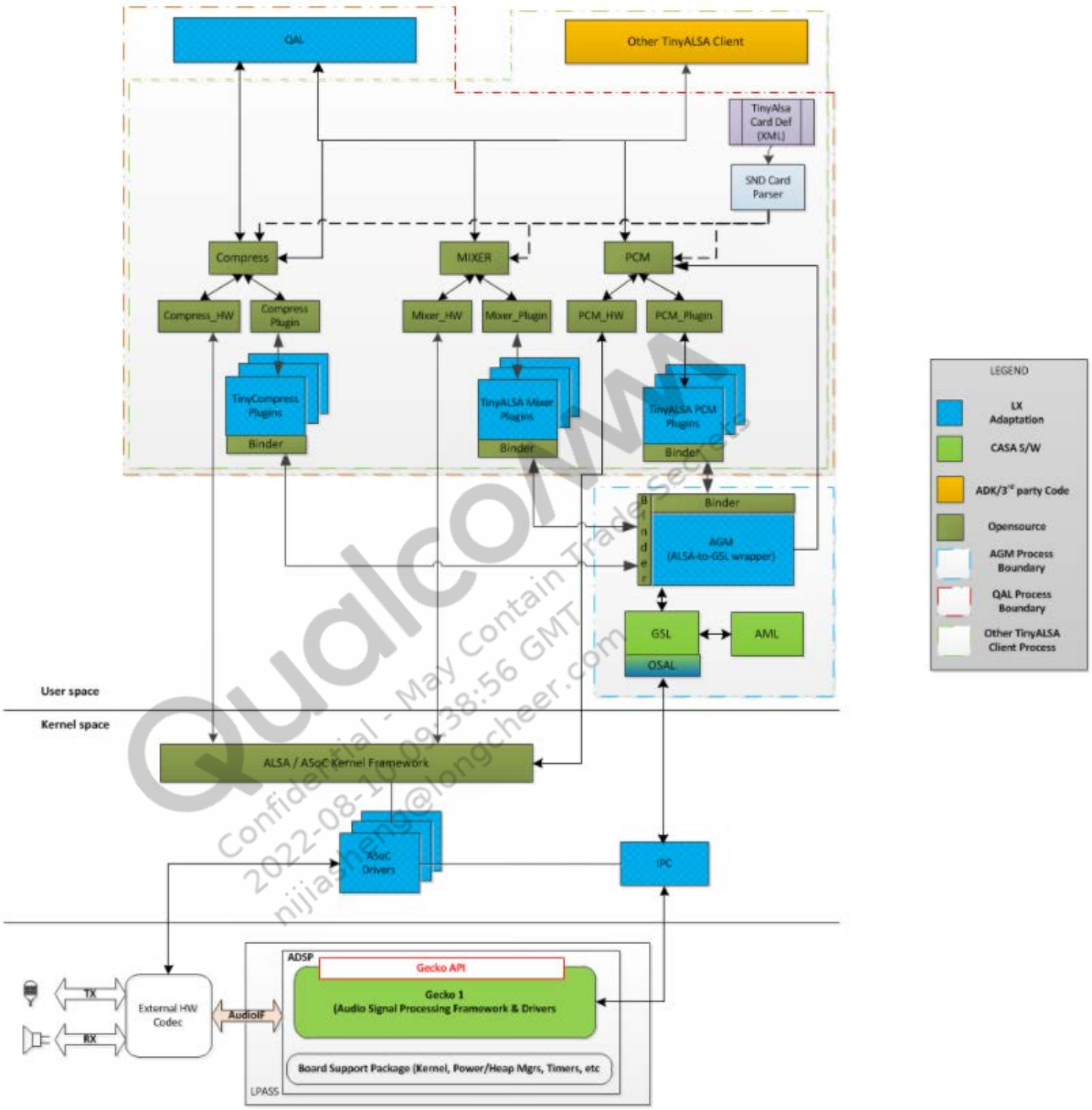
In the AudioReach Linux Android adaptation, the QAL is the client of the TinyALSA Layer. Whenever QAL calls the mixer control, tinypcm, or tinycompress APIs, the APIs are handled by the AGM. The AGM parses the information passed in the mixer controls and uses this information to get use case-related graph information from the ACDB using AML APIs. AGM then uses the information to call GSL APIs to instantiate the graph in the ADSP. The AGM can also handle the PCM and compress API calls and exchange data with the ADSP via the GSL APIs .

The TinyALSA framework uses plugins to expose virtual PCM devices that are defined in card-defs.xml. This configuration file is parsed by the QAL

To verify audio use cases (for example, playback, recording, audio loopback) only by calling AGM APIs, utilities such as agmplay, agmmixer, and agmcompressplay can be used. These utilities are also helpful in audio use case verification during the initial stage of board bringup.

Qualcomm Confidential May Contain Trade Secrets 2022-08-10 04:33:06 GMT nijiasheng@longsheer.com

The following diagram shows the TinyALSA plugin architecture.



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:56 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

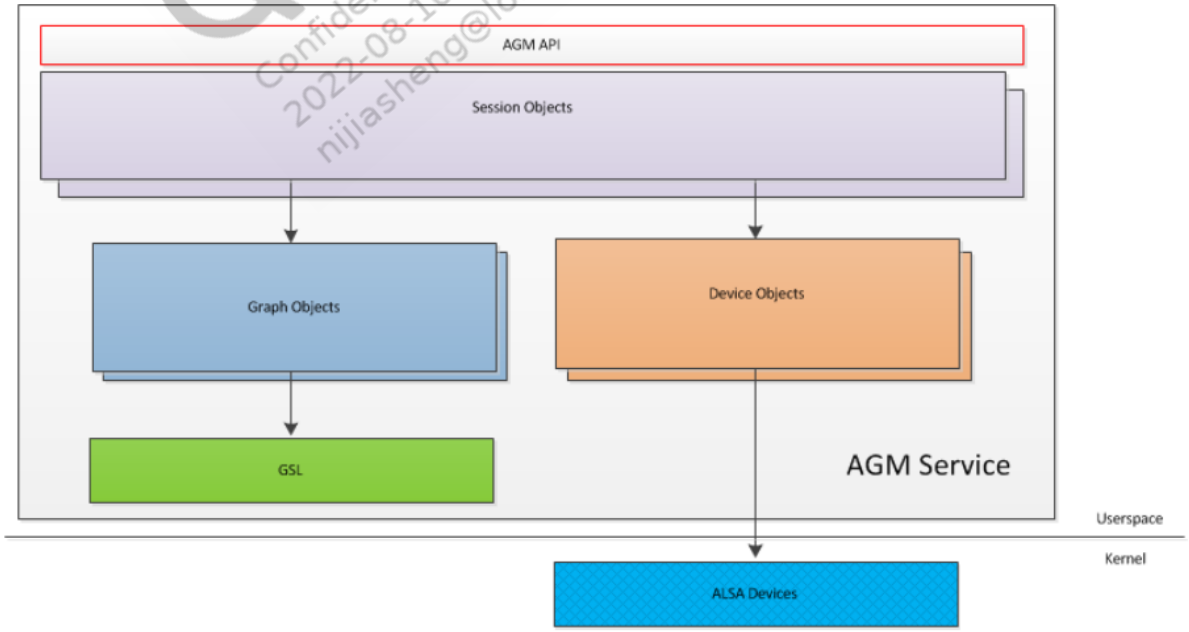
The following table provides a list of available mixer controls.

Purpose	Mixer control
Frontend – Backend	‘PCM0 connect’ SLIM_0_RX 'PCM0 disconnect' SLIM_0_RX
Backend fixup	'SLIM_0_RX rate ch fmt' <sample rate> <channels> <fmt>
GKV/CKV	’SLIM_0_RX metadata' bytes 'PCM0 metadata' bytes
Update CKV	'PCM0 setcal’ bytes
Set module(s) custom parameter(s)	'PCM0 setParam' bytes 'PCM0 getParam' bytes
Set module(s) parameter using TKV	'PCM0 SetParamTag' bytes
Echo reference	'PCM1 echoReference’ SLIM_0_RX
Setup loopback	'PCM1 loopback’ PCM0
Sidetone for voice	'PCM0 Sidetone’ PCM1
Register for events/get event payload	'PCM0 event’ bytes

3.5 AGM

The AGM is an audio service that runs in the Linux user space. AGM provides interfaces to allow TinyALSA-based mixer controls as well as PCM and compressed plugins to interact and enable various audio use cases.

The following diagram shows the various blocks in the AGM service.



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:50 GMT nijiasheng@longcheer.com

3.5.1 Session object

A session object represents an audio playback or capture session. A session object is created whenever a mixer control specific to a session is invoked or session-specific APIs are invoked. Once created, the session object is added to session pool maintained by the AGM service.

Session objects have the following roles and responsibilities:

- Provide APIs to TinyALSA plugins to realize stream configuration
- Cache stream, stream device, device-specific metadata
- Create the graph and device object and provide required information to these objects
- Manage state transition of graph and device object

3.5.2 Graph object

A graph object represents a graph; an interconnection of modules running in SPF. Graph objects interact with the GSL to open, manage, and close graphs in SPF.

Graph objects have the following roles and responsibilities:

- Provide APIs for creating, adding and removing graphs. Should mirror GSL APIs.
- Interact with GSL to create and manage graphs on SPF
- Configure the following modules to enable ALSA APIs:
  - Stream – Sharedmem, decoder/encoder (PCM and compressed), PCM converter
  - Device – Hardware EPs

3.5.3 Device object

A device object represents an ALSA device as enumerated by the ALSA sound card. Each ALSA device is a direct backend DAI link that sets up kernel resources such as GPIOs, clocks, and the codec driver.

Device objects have the following roles and responsibilities:

- Enumerates available audio interfaces by reading the ALSA sound card
- Provides device APIs for transitioning device states
- Manage ALSA backend states with ref count. The ALSA backend is configured with a client-specified media configuration.
- Cache metadata (GKV and CKV) for the device

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 03:38:56 GMT lijiasheng@longshee.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.5.4 AGM service

AGM runs as an independent service for the following reasons:

- Frontend-to-backend connections
  - In a kernel based ALSA implementation, the ASoC framework in the kernel provides the capability to manage dynamic connections between ALSA frontends and ALSA backend AGMs. The ASoC frameworks manages the refcount of ALSA frontend to ALSA backend connections. In a multi-ALSA frontend to single ALSA backend connection, tearing down of one ALSA frontend connected to an ALSA backend will not impact the other ALSA frontend connected use cases.
  - The TinyALSA library and plugins are loaded in the calling application's context. Hence, each application is not aware of other client's hardware resource utilization. In the TinyALSA plugin-based design, the AGM provides the capability to manage multiple frontend-to-backend connections. The AGM runs as a service so that it can cater to multiple ALSA clients to set up use cases and handle management of frontend-to-backend connections.
- GSL and ACDB software libraries – GSL and ACDB software libraries are singleton libraries. These libraries need to be run from a single process context to ensure that they are not loaded to multiple clients. Since AGM is run as an independent process, it loads GSL which in turn loads the ACDB library.

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:56 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.5.5 Tags in AGM

The following table describes the tags in the AGM.

Tag name	Module	TKV-based configuration	Runtime configuration in code
DEVICE_HW_ENDPOINT_RX	Rx hardware EP only	Interface type-specific configuration based on channel count as TKV. Excludes dynamic EPs such as DP, HDMI, and USB as these are configured by QAL directly.	Media config configuration
DEVICE_HW_ENDPOINT_TX	TX hardware EP only	Interface type-specific configuration with TKV as channel count. Excludes dynamic EPs such as DP, HDMI, and USB as these are configured by QAL directly.	Media config configuration
TAG_STREAM_PLACEHOLDER_DECODER	Place holder decoder	Real module ID with TKV MEDIA_FMT_ID	Output media format
TAG_STREAM_PLACEHOLDER_ENCODER	Place holder encoder	Real module ID with TKV MEDIA_FMT_ID	Output media format
STREAM_PCM_DECODER	PCM decoder	None	Output media format
STREAM_PCM_ENCODER	PCM encoder	None	Output media format
STREAM_PCM_CONVERTER	PCM converter	None	Output media format
STREAM_INPUT_MEDIA_FORMAT	SharedMem EP	None	Input media format
TAG_STREAM_SPR	SPR in stream	None	SPR_DELAY_PATH end point. Hardware EP ID determined by using the DEVICE_HW_ ENDPOINT_RX/TX tag. Retrieves session timestamp.
TAG_PAUSE	Stream pause module	None	PAUSE start/resume

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:56 GMT nijishen@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.6 AML

The AML exposes APIs to allow clients to get binary blobs for graph components (for example, subgraphs, components, modules, edges etc.). AML APIs can be called from either GSL and QAL.

QAL interacts directly with the GSL in the following cases:

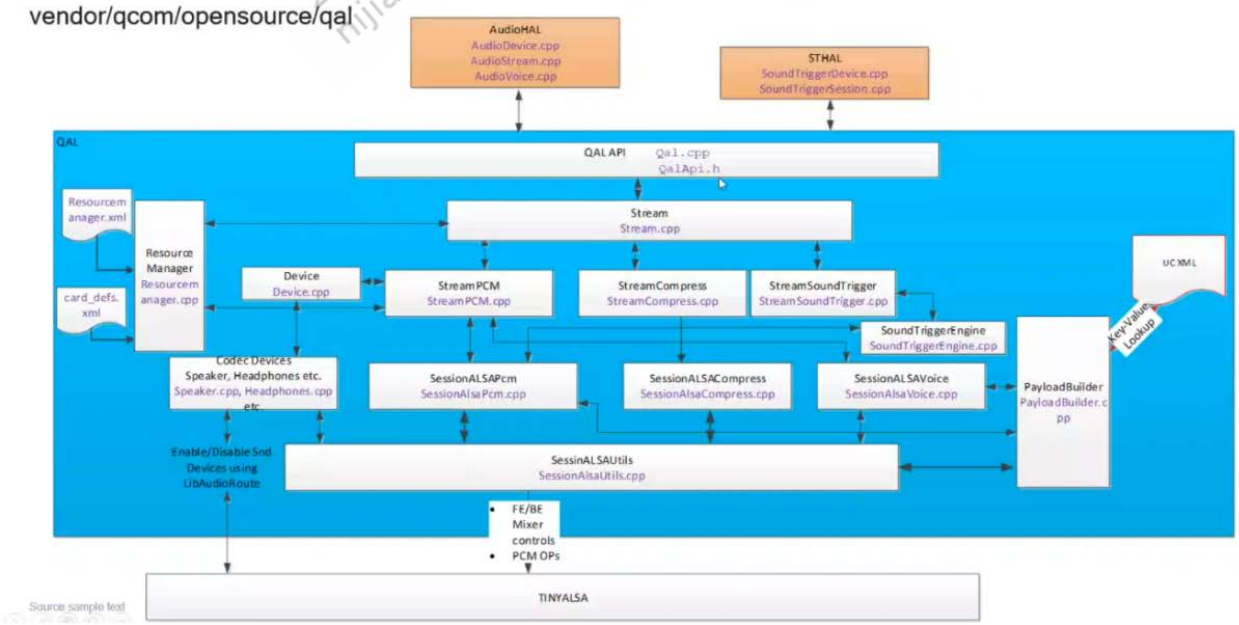
- Get custom parameters for H2XML based configuration – This operation requires querying certain module-related calibration parameters to form a calibration packet the SPF can understand. This is useful for UI custom module configurations and other frameworks (for example, the generic effects frameworks (GEF) on Linux Android).
- Device management and control within QAL can directly call AML APIs to retrieve codec calibration data.

3.7 QAL

QAL has the following responsibilities:

- Interface with clients to set up audio and voice use cases
- Map stream/device(s) parameters to GKVs/CKVs
- Send mixer controls to the frontend and backend with GKVs/CKVs
- Open/close PCM and compress device nodes with TinyALSA APIs
- Configure modules in the graph pertaining to the session using module tags (TKVs)
- Provide APIs to enable use cases across LX platforms

The following diagram shows the QAL architecture.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

QAL is responsible for representing the use case in the form a graph. A graph is identified as a GKV.

QAL uses the information in configuration files (for example, resourcemanager.xml, kvh2xml.h, and card-defs.xml) to send the following information by calling TinyALSA APIs:

- Frontend-to-backend routing
- Backend configuration (if there is a need to overwrite the default configuration)
- GKV and CKV
- Update the CKV
- Set module(s) custom parameter(s)
- Echo reference
- Loopback set up
- Sidetone for voice
- Register for events/get event payload

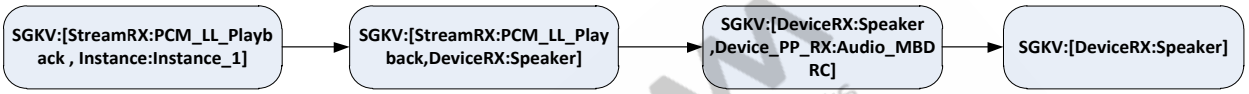
QAL also calls tinypcm and tinycompress APIs for playback and capture use cases and some other special cases.

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:56 GMT nijiasheng@longcheer.com

4 Audio use cases

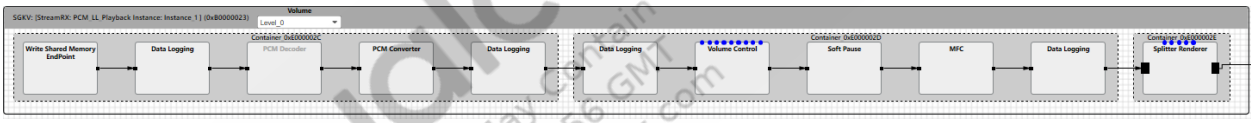
4.1 Playback

4.1.1 Graph

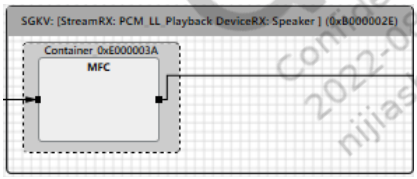


Where:

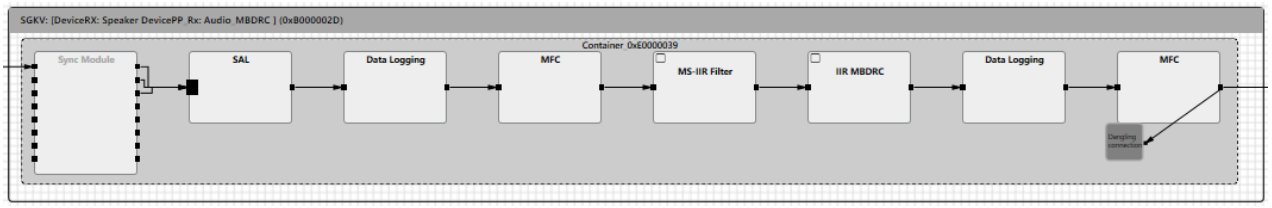
SGKV:[StreamRX:PCM_LL_Playback , Instance:Instance_1]



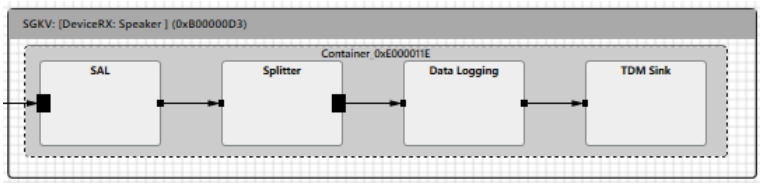
SGKV:[StreamRX:PCM_LL_Playback,DeviceRX:Speaker]



SGKV:[DeviceRX:Speaker ,Device_PP_RX:Audio_MBDRC]

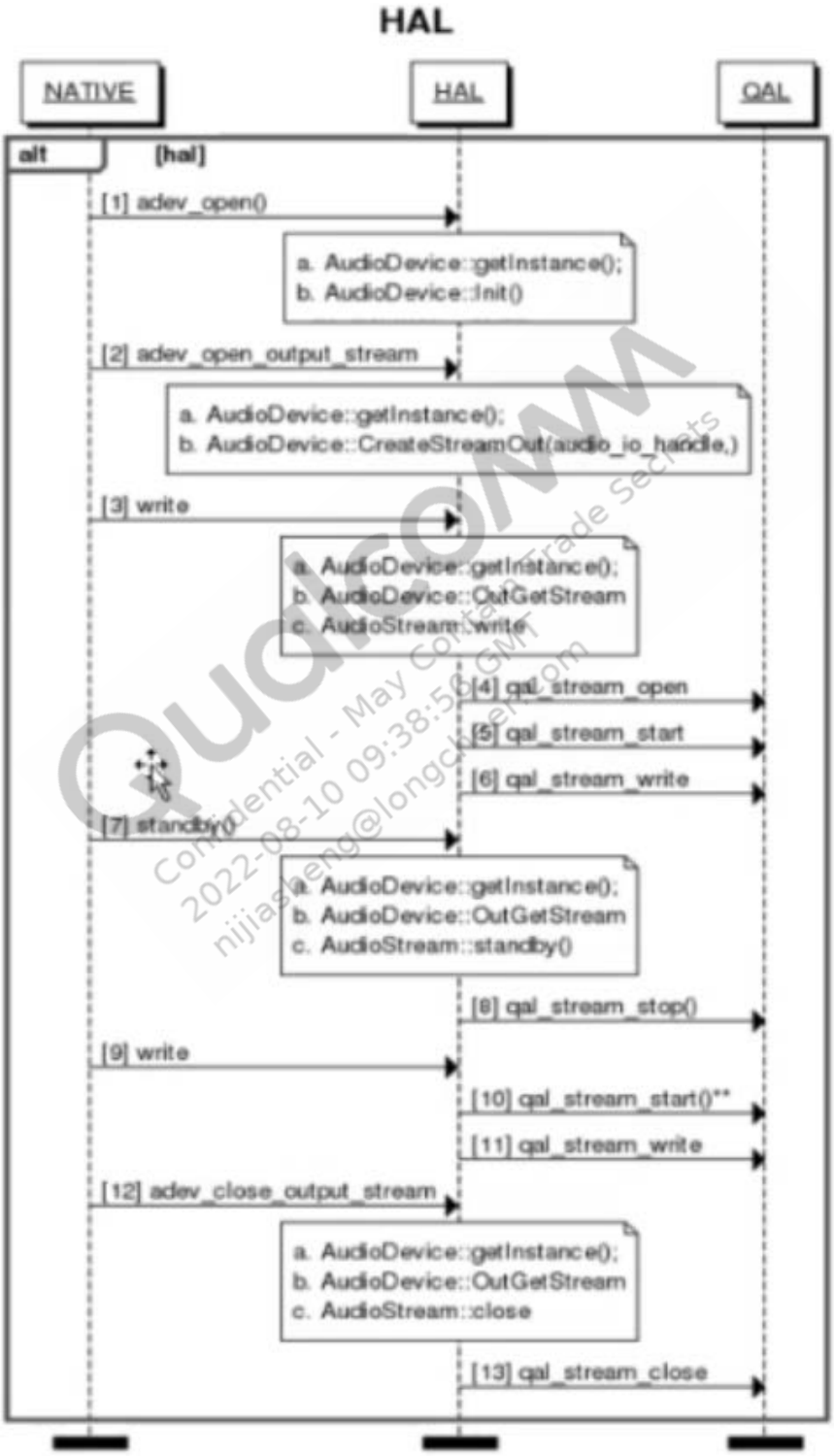


SGKV:[DeviceRX:Speaker]



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:56 GMT nijiasheng@longsheer.com

4.1.2 High-level call flow



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:58 GMT nijiasheng@longcheer.com

4.1.3 Logcat logs

Start sequence

```
//Audio hal calls  qal_stream_open when doing the First write operation
07-23 16:14:24.358   828  3371 I QAL:    : qal_stream_open:129 Enter.
 . . .
07-23 16:14:24.359   828  3371 I QAL: Device: setDeviceAttributes:221
DeviceAttributes for Device Id 3 updated
07-23 16:14:24.359   828  3371 E QAL: Stream: create:149 stream
0xf50c2da0K created
07-23 16:14:24.360   828  3371 E QAL: Stream: getAssociatedDevices:253
no. of devices 1
07-23 16:14:24.360   828  3371 E QAL: ResourceManager:
getBackEndNames:3106 getBackEndNames (RX): CODEC_DMA-LPAIF_WSA-RX-0
//Call the resource Manager APIs  to get the Front End PCM ID and the
Back End name  .

07-23 16:14:24.360   828  3371 I QAL: ResourceManager:
allocateFrontEndIds:2792 allocateFrontEndIds: front end 110
07-23 16:14:24.360   828  3371 E QAL: Stream: getAssociatedDevices:253
no. of devices 1

 . . .
//StreamPCMobject is created
07-23 16:14:24.360   828  3371 E QAL: SessionAlsaUtils:
getFeMixerControl:266 mixer control PCM110 control
07-23 16:14:24.360   828  3371 E QAL: SessionAlsaUtils:
getFeMixerControl:266 mixer control PCM110 metadata
07-23 16:14:24.361   828  3371 E QAL: SessionAlsaUtils:
getFeMixerControl:266 mixer control PCM110 connect
// mixer controls are dynamically created

07-23 16:14:24.361   828  3371 D PLUGIN: mixer: amp_pcm_mtd_control_put:
864 amp_pcm_mtd_control_put: value = 0
07-23 16:14:24.361   828  3371 D PLUGIN: mixer: amp_pcm_metadata_put:
932 amp_pcm_metadata_put: enter
07-23 16:14:24.361   896  1852 D AGM: metadata: metadata_print: 77
************************Metadata************************
07-23 16:14:24.362   896  1852 D AGM: metadata: metadata_print: 78 GKV
size:2
07-23 16:14:24.362   896  1852 D AGM: metadata: metadata_print: 81
key:0xa1000000, value:0xa100000e
07-23 16:14:24.362   896  1852 D AGM: metadata: metadata_print: 81
key:0xab000000, value:0x1
// {STREAMRX :  PCM_LL_PLAYBACK  , INSTANCE : 1}

07-23 16:14:24.362   896  1852 D AGM: metadata: metadata_print: 85
07-23 16:14:24.362   896  1852 D AGM: metadata: CKV count:1
```

```
07-23 16:14:24.362   896  1852 D AGM: metadata: metadata_print: 88
key:0xa4000000, value:0x15
//{VOLUME : 0x15}

07-23 16:14:24.363   896  1852 D AGM: metadata: metadata_print: 90
07-23 16:14:24.363   896  1852 D AGM: metadata: metadata_print: 92
Property ID:134217744  (0x80000010)  comes from
/gsl/api/gsl_subgraph_platform_driver_props.xml
07-23 16:14:24.363   896  1852 D AGM: metadata: metadata_print: 93
property count:1
07-23 16:14:24.363   896  1852 D AGM: metadata: metadata_print: 96
Property Values:
07-23 16:14:24.363   896  1852 D AGM: metadata: metadata_print: 98 0x1
07-23 16:14:24.363   896  1852 D AGM: metadata: metadata_print: 100
07-23 16:14:24.363   896  1852 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
07-23 16:14:24.363   896  1852 D AGM: metadata:
07-23 16:14:24.364   828  3371 E QAL: Stream: getAssociatedDevices:253
no. of devices 1
07-23 16:14:24.364   828  3371 D PLUGIN: mixer: amp_be_metadata_put: 777
amp_be_metadata_put: enter
07-23 16:14:24.365   896  6092 D AGM: metadata: metadata_print: 77
*************************Metadata*************************
07-23 16:14:24.365   896  6092 D AGM: metadata: metadata_print: 78 GKV
size:1
07-23 16:14:24.365   896  6092 D AGM: metadata: metadata_print: 81
key:0xa2000000, value:0xa2000001
{DEVICERX:SPEAKER }
07-23 16:14:24.365   896  6092 D AGM: metadata: metadata_print: 85
07-23 16:14:24.365   896  6092 D AGM: metadata: CKV count:0
07-23 16:14:24.365   896  6092 D AGM: metadata: metadata_print: 90
07-23 16:14:24.365   896  6092 D AGM: metadata: metadata_print: 92

07-23 16:14:24.365   896  6092 D AGM: metadata: metadata_print: 93
property count:1
07-23 16:14:24.366   896  6092 D AGM: metadata: metadata_print: 96
Property Values:
07-23 16:14:24.366   896  6092 D AGM: metadata: metadata_print: 98 0x2
07-23 16:14:24.366   896  6092 D AGM: metadata: metadata_print: 100
07-23 16:14:24.366   896  6092 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
07-23 16:14:24.366   896  6092 D AGM: metadata:
07-23 16:14:24.366   828  3371 D PLUGIN: mixer: amp_pcm_mtd_control_put:
864 amp_pcm_mtd_control_put: value = 1
07-23 16:14:24.367   828  3371 D PLUGIN: mixer: amp_pcm_metadata_put:
932 amp_pcm_metadata_put: enter
07-23 16:14:24.367   896  1851 D AGM: metadata: metadata_print: 77
*************************Metadata*************************
```

Qualcomm Confidential - May Contain Trade Secrets 2022-08-09 09:38:56 GMT nijisheng@ensrcheer.com

```
07-23 16:14:24.367   896  1851 D AGM: metadata: metadata_print: 78 GKV
size:2
07-23 16:14:24.367   896  1851 D AGM: metadata: metadata_print: 81
key:0xac000000, value:0xac000002
07-23 16:14:24.367   896  1851 D AGM: metadata: metadata_print: 81
key:0xa2000000, value:0xa2000001
{DEVICEPP_RX:DEVICEPP_RX_AUDIO_MBDRC ,DEVICERX:SPEAKER}
07-23 16:14:24.367   896  1851 D AGM: metadata: metadata_print: 85
07-23 16:14:24.367   896  1851 D AGM: metadata: CKV count:0
07-23 16:14:24.368   896  1851 D AGM: metadata: metadata_print: 90
07-23 16:14:24.368   896  1851 D AGM: metadata: metadata_print: 92
07-23 16:14:24.368   896  1851 D AGM: metadata: metadata_print: 93
property count:1
07-23 16:14:24.368   896  1851 D AGM: metadata: metadata_print: 96
Property Values:
07-23 16:14:24.368   896  1851 D AGM: metadata: metadata_print: 98 0x3
07-23 16:14:24.368   896  1851 D AGM: metadata: metadata_print: 100
07-23 16:14:24.369   896  1851 D AGM: metadata: metadata_print: 101
****************Metadata Done****************************
07-23 16:14:24.369   896  1851 D AGM: metadata:
07-23 16:14:24.369   828  3371 D PLUGIN: mixer: amp_pcm_aif_connect_put:
808 amp_pcm_aif_connect_put: enter
07-23 16:14:24.369   828  3371 I QAL: SessionAlsaUtils:
setDeviceMediaConfig:663 CODEC_DMA-LPAIF_WSA-RX-0 rate ch fmt data_fmt
48000 1 2 1
07-23 16:14:24.370   828  3371 D PLUGIN: mixer: amp_be_media_fmt_put:
695 amp_be_media_fmt_put: enter
07-23 16:14:24.370   828  3371 I QAL: ResourceManager: getInstance:2590
Enter.
07-23 16:14:24.370   828  3371 I QAL: ResourceManager: getInstance:2598
Exit.
07-23 16:14:24.371   828  3371 E QAL: Stream: setBufInfo:338 no of buf 2
and send buf 3c0
// Number of buffers = 2  , size of buffer  = 960  =  20ms  for 48Khz

07-23 16:14:24.375   828  3371 I QAL: SessionAlsaUtils:
setDeviceMediaConfig:663 CODEC_DMA-LPAIF_WSA-RX-0 rate ch fmt data_fmt
48000 1 2 1
07-23 16:14:24.375   828  3371 D PLUGIN: mixer: amp_be_media_fmt_put:
695 amp_be_media_fmt_put: enter
07-23 16:14:24.376   828  3371 I QAL: ResourceManager: getInstance:2590
Enter.
07-23 16:14:24.376   828  3371 I QAL: ResourceManager: getInstance:2598
Exit.
07-23 16:14:24.376   828  3371 I QAL: ResourceManager: getInstance:2590
Enter.
07-23 16:14:24.376   828  3371 I QAL: ResourceManager: getInstance:2598
Exit.
```

Qualcomm Confidential - May Contain Trade Secrets 2022-08-11 09:38:56 GMT nijsherdesonicher.com

```
07-23 16:14:24.376   828  3371 D PLUGIN: mixer: amp_pcm_mtd_control_put:
864 amp_pcm_mtd_control_put: value = 1
07-23 16:14:24.376   828  3371 D PLUGIN: mixer: amp_pcm_tag_info_get:
1156 amp_pcm_tag_info_get: enter

07-23 16:14:24.387   828  3371 E agm_client_wrapper: agm_session_open
called with handle = f1a64838 , *handle = 0
07-23 16:14:24.389   896  6092 D AGM: graph: graph_open: 425 entry
07-23 16:14:24.394   896  6092 D AGM: graph: graph_open: 492 miid 40bd
mid 7001000 tag c0000002   // miid autogenerated by QACT, mid:Write
Shared Memory EndPoint , tag:STREAM_INPUT_MEDIA_FORMAT
07-23 16:14:24.394   896  6092 D AGM: graph: graph_open: 537 miid 43b1
mid 7001023 tag c0000004   // miid autogenerated by QACT, mid:Splitter
Renderer, tag:DEVICE_HW_ENDPOINT_RX
07-23 16:14:24.394   896  6092 D AGM: graph: graph_open: 492 miid 40c4
mid 7001019 tag c0000006  // miid autogenerated by QACT, mid:Soft
Pause , tag:TAG_PAUSE
07-23 16:14:24.394   896  6092 D AGM: graph: graph_open: 492 miid 40bf
mid 7001005 tag c000000e // miid autogenerated by QACT, mid:PCM
decoder , tag: STREAM_PCM_DECODER
07-23 16:14:24.394   896  6092 D AGM: graph: graph_open: 492 miid 40c0
mid 7001003 tag c0000010  // miid autogenerated by QACT, mid:PCM
Converter , tag:STREAM_PCM_CONVERTER
07-23 16:14:24.394   896  6092 D AGM: graph: graph_open: 492 miid 4165
mid 7001032 tag c0000013  // miid autogenerated by QACT, mid:Write
Splitter and Renderer , tag: STREAM_SPR
07-23 16:14:24.399   896  6092 D AGM: graph: graph_open: 570
graph_handle 47534c
07-23 16:14:24.399   828  3371 E agm_client_wrapper: agm_session_open
Received handle = f1a64838 , *handle = 0
07-23 16:14:24.399   828  3371 D PLUGIN: pcm: agm_pcm_hw_params: 364
agm_pcm_hw_params: mode: 0
07-23 16:14:24.399   828  3371 E agm_client_wrapper:
agm_session_set_config called with handle = 83f65710
07-23 16:14:24.401   828  3371 E agm_client_wrapper:
agm_session_set_config called with handle = 83f65710
07-23 16:14:24.402   828  3371 E QAL: Stream: getAssociatedDevices:253
no. of devices 1
07-23 16:14:24.402   828  3371 E QAL: Stream: getAssociatedDevices:253
no. of devices 1
07-23 16:14:24.402   828  3371 I QAL: ResourceManager: getInstance:2590
Enter.
07-23 16:14:24.402   828  3371 I QAL: ResourceManager: getInstance:2598
Exit.
07-23 16:14:24.402   828  3371 I QAL: ResourceManager: getInstance:2590
Enter.
07-23 16:14:24.403   828  3371 I QAL: ResourceManager: getInstance:2598
Exit.
07-23 16:14:24.403   828  3371 D PLUGIN: mixer: amp_pcm_mtd_control_put:
864 amp_pcm_mtd_control_put: value = 1
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
07-23 16:14:24.403   828  3371 D PLUGIN: mixer: amp_pcm_tag_info_get:
1156 amp_pcm_tag_info_get: enter
07-23 16:14:24.404   828  3371 I QAL: Session: updateCustomPayload:264
customPayloadSize = 32
07-23 16:14:24.405   828  3371 I QAL: ResourceManager: getInstance:2590
Enter.
07-23 16:14:24.405   828  3371 I QAL: ResourceManager: getInstance:2598
Exit.
07-23 16:14:24.405   828  3371 D PLUGIN: mixer: amp_pcm_set_param_put:
1037 amp_pcm_set_param_put: enter
07-23 16:14:24.405   896  6092 D AGM: graph: graph_set_config: 878 entry
graph_handle 47534c
07-23 16:14:24.444   896  6092 D AGM: graph: graph_prepare: 650 entry
graph_handle 47534c
07-23 16:14:24.444   896  6092 D AGM: graph_module:
configure_pcm_shared_mem_ep: 1137 entry mod tag c0000002 miid 40bd mid
7001000
07-23 16:14:24.451   896  6092 D AGM: graph_module:
configure_pcm_shared_mem_ep: 1208 exit
07-23 16:14:24.451   896  6092 D AGM: graph_module:
configure_hw_ep_media_config: 609 entry mod tag c0000004 miid 43b1 mid
7001023
07-23 16:14:24.451   896  6092 E AGM: graph_module:
configure_hw_ep_media_config: 639 rate 48000 bw 16 ch 1, data_fmt 1
07-23 16:14:24.453   896  6092 D AGM: graph_module:
configure_hw_ep_media_config: 649 exit
07-23 16:14:24.453   896  6092 D AGM: graph_module:
configure_codec_dma_ep: 236 cdc_dma intf cfg lpaif 2 indx 1 ch_msk 2
07-23 16:14:24.453   896  6092 D AGM: graph_module:
configure_codec_dma_ep: 251 exit
07-23 16:14:24.454   896  6092 D AGM: graph_module:
configure_output_media_format: 827 exit
07-23 16:14:24.454   896  6092 D AGM: graph_module:
configure_output_media_format: 827 exit
07-23 16:14:24.454   896  6092 D AGM: graph_module: configure_spr: 1241
SPR module IID 4165
07-23 16:14:24.454   896  6092 D AGM: graph_module: configure_spr: 1264
HW EP module IID 43b1
07-23 16:14:24.455   896  6092 D AGM: graph: configure_buffer_params:
156 Playback sess buf_sz 960 num_bufs 2
07-23 16:14:24.455   896  6092 E AGM: graph: configure_buffer_params:
184 Unsupported buffer mode : 0, Default to Blocking
07-23 16:14:24.455   896  6092 D AGM: graph: configure_buffer_params:
206 exit
07-23 16:14:24.457   896  6092 D AGM: graph: graph_prepare: 721 exit
07-23 16:14:24.458   896  6092 D AGM: graph: graph_start: 735 entry
graph_handle 47534c
07-23 16:14:24.461   896  6092 D AGM: graph: graph_start: 747 exit
```


MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
07-23 16:14:24.461   828  3371 I QAL: ResourceManager: getInstance:2590
Enter.
07-23 16:14:24.462   828  3371 I QAL: ResourceManager: getInstance:2598
Exit.
07-23 16:14:24.465   896  6092 E ACDB    :
AcdbCmdGetSubgraphCalDataNonPersist:3759 Error[19]: Unable to retrieve
calibration data for Subgraph(b0000002). Skipping..
07-23 16:14:24.465   896  6092 E ACDB    :
AcdbCmdGetSubgraphCalDataNonPersist:3759 Error[19]: Unable to retrieve
calibration data for Subgraph(b0000024). Skipping..
 . . .
07-23 16:14:24.470   828  3371 V ahal_AudioDevice: OutGetStream:
stream_out(0xf50c2260)
07-23 16:14:24.470   828  3371 V ahal_AudioDevice: OutGetStream: Found
stream associated with stream_out
07-23 16:14:24.470   828  3371 V ahal_AudioDevice: OutGetStream:
astream_out(0xf50c2260)
```

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:56 GMT nijiasheng@longcheer.com

Stop sequence

```
07-23 16:14:27.671   828  2345 I QAL:    : qal_stream_stop:228 Enter.
Stream handle :0xf50c2da0K
07-23 16:14:27.671   828  2345 E QAL: StreamPCM: stop:461 Enter. session
handle - 0xf44c0c50K mStreamAttr->direction - 1 state 3

07-23 16:14:27.671   896  6092 D AGM: graph: graph_stop: 763 entry
graph_handle 47534c
07-23 16:14:27.674   896  6092 D AGM: graph: graph_stop: 796 exit
07-23 16:14:27.675   828  2345 E QAL: Stream: getAssociatedDevices:253
no. of devices 1


07-23 16:14:27.688   828  2345 I QAL:    : qal_stream_stop:236 Exit.
status 0
07-23 16:14:27.689   828  2345 I QAL:    : qal_stream_close:176 Enter.
Stream handle :0xf50c2da0K


07-23 16:14:27.689   828  2345 I QAL: StreamPCM: close:209 Enter.
session handle - 0xf44c0c50K device count - 1 state 5
07-23 16:14:27.690   828  1848 D ahal_AudioDevice: GetParameters: exit:
returns -
07-23 16:14:27.690   828  2345 E QAL: Stream: getAssociatedDevices:253
no. of devices 1

07-23 16:14:27.691   828  2345 E QAL: SessionAlsaPcm: close:900
backendname CODEC_DMA-LPAIF_WSA-RX-0
07-23 16:14:27.691   828  2345 E QAL: SessionAlsaUtils:
getFeMixerControl:266 mixer control PCM110 control
07-23 16:14:27.691   828  2345 E QAL: SessionAlsaUtils:
getFeMixerControl:266 mixer control PCM110 metadata
07-23 16:14:27.691   828  2345 E QAL: SessionAlsaUtils:
getFeMixerControl:266 mixer control PCM110 connect
07-23 16:14:27.692   828  2345 E QAL: SessionAlsaUtils:
getFeMixerControl:266 mixer control PCM110 disconnect
// Mixer controls to disconnect the graph


07-23 16:14:27.692   828  2345 D PLUGIN: mixer: amp_pcm_metadata_put:
932 amp_pcm_metadata_put: enter
07-23 16:14:27.692   896  6092 D AGM: metadata: metadata_print: 77
*************************Metadata************************
07-23 16:14:27.692   896  6092 D AGM: metadata: metadata_print: 78 GKV
size:0
07-23 16:14:27.692   896  6092 D AGM: metadata: metadata_print: 85
07-23 16:14:27.692   896  6092 D AGM: metadata: CKV count:0
07-23 16:14:27.692   896  6092 D AGM: metadata: metadata_print: 90
```

Qualcomm Confidential – May Contain Trade Secrets 2023-08-10 09:38:56 GMT mike.hong@longcheer.com

```
07-23 16:14:27.692   896  6092 D AGM: metadata: metadata_print: 92
Property ID:134217744
07-23 16:14:27.692   896  6092 D AGM: metadata: metadata_print: 93
property count:1
07-23 16:14:27.692   896  6092 D AGM: metadata: metadata_print: 96
Property Values:
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 98 0x1
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 100
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
07-23 16:14:27.693   896  6092 D AGM: metadata:
07-23 16:14:27.693   828  2345 D PLUGIN: mixer: amp_be_metadata_put: 777
amp_be_metadata_put: enter
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 77
*************************Metadata*************************
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 78 GKV
size:0
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 85
07-23 16:14:27.693   896  6092 D AGM: metadata: CKV count:0
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 90
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 92
Property ID:134217744
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 93
property count:1
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 96
Property Values:
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 98 0x2
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 100
07-23 16:14:27.693   896  6092 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
07-23 16:14:27.693   896  6092 D AGM: metadata:
07-23 16:14:27.693   828  2345 D PLUGIN: mixer: amp_pcm_mtd_control_put:
864 amp_pcm_mtd_control_put: value = 1
07-23 16:14:27.693   828  2345 D PLUGIN: mixer: amp_pcm_metadata_put:
932 amp_pcm_metadata_put: enter
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 77
*************************Metadata*************************
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 78 GKV
size:0
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 85
07-23 16:14:27.694   896  6092 D AGM: metadata: CKV count:0
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 90
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 92
Property ID:134217744
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 93
property count:1
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 96
Property Values:
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 98 0x3
```

Qualcomm Confidential – May Contain Trade Secrets 2022-08-10 09:08:58 GMT nin.sheng@qualcomm.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

```
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 100
07-23 16:14:27.694   896  6092 D AGM: metadata: metadata_print: 101
****************Metadata Done***************************
07-23 16:14:27.694   896  6092 D AGM: metadata:


07-23 16:14:27.694   828  2345 D PLUGIN: mixer: amp_pcm_aif_connect_put:
808 amp_pcm_aif_connect_put: enter
07-23 16:14:27.694   896  1851 D AGM: graph: graph_stop: 763 entry
graph_handle 47534c
07-23 16:14:27.694   896  1851 E AGM: graph: graph_stop: 765 graph
object is already in STOP state
07-23 16:14:27.694   896  1851 D AGM: graph: graph_stop: 796 exit
07-23 16:14:27.695   896  1852 D AGM: graph: graph_close: 606 entry
handle 47534c
07-23 16:14:27.696   896  1852 D AGM: graph: graph_close: 626 exit
07-23 16:14:27.696   896  1852 E AGM: session: session_close: 1076 exit
07-23 16:14:27.697   828  2345 I QAL: ResourceManager:
freeFrontEndIds:2943 stream type 1, freeing 110
07-23 16:14:27.697   828  2345 E QAL: ResourceManager:
deregisterStream:1461 stream type 1
07-23 16:14:27.697   828  2345 I QAL: StreamPCM: close:264 Exit. closed
the stream successfully 0 status 0
07-23 16:14:27.697   828  2345 I QAL: ResourceManager: getInstance:2590
Enter.
07-23 16:14:27.697   828  2345 I QAL: ResourceManager: getInstance:2598
Exit.
07-23 16:14:27.698   828  2345 I QAL:    : qal_stream_close:193 Exit.
status 0
```

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:38:56 GMT nijiasheng@lonwcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A References

A.1 Acronyms and terms

Acronym or term	Definition
ACDB	Audio calibration database
ADSP	Application DSP
AGM	Audio graph manager
APM	Audio process manager
CKV	Calibration key vector
EP	Endpoint
GEF	Generic effects framework
GKV	Graph key vector
GPR	Graph packet router
GSL	Graph service layer
IPC	Inter-processor communication
SPF	Signal processing framework
SSL	SPF service layer
TKV	Tag key vector

Qualcomm Confidential – May Contain Trade Secrets
2022-03-04 03:43:56 GMT
nijiasheng@lonestreer.com